Git & GitHub – Complete Teaching Topics

Module 1: Introduction to Git & GitHub

1. What is Git?

Students should understand:

- What is Version Control?
- Why do programmers need version control?
- Problems without Git
- What Git can do
- Advantages of Git
- Git in software development
- Real-life example of version history

Simple example: A student creates `project.py`, changes it several times, and wants to return to an older version. Git helps manage these versions.

2. What is GitHub?

Teach:

- What is GitHub?
- Why GitHub is used
- GitHub repositories
- Storing projects online
- Sharing projects
- Collaboration
- Open-source projects
- GitHub as a developer portfolio

Simple explanation:

Git is a tool used on your computer to track changes.

GitHub is an online platform where Git repositories can be stored, shared and collaborated on.

3. Git vs GitHub

Git	GitHub
Software/tool	Online platform
Works mainly on local computer	Cloud-based service
Tracks changes	Hosts Git repositories
Works from Command Prompt/Terminal	Works through website + Git
Can work without GitHub	Uses Git repositories

Student activity: Ask students to explain Git and GitHub in their own words.

Module 2: Installing and Configuring Git

4. Installing Git

Teach:

- Downloading Git
- Installing Git on Windows
- Git Bash
- Git Command Prompt
- Checking installation

Command:

```
git --version
```

Practical:
Students verify that Git is correctly installed.

Git Configuration

Teach:

- Why configuration is required
- Username
- Email
- Checking configuration

Commands:

```
git config --global user.name "Student Name"
git config --global user.email "student@example.com"
```

Check:

```
git config --list
```

Explain that Git uses this identity when creating commits.

Module 3: First Local Git Repository

6. `git init`

Explain:

- What is a repository?
- Local repository
- Initializing a project
- `.git` folder
- Why `.git` is important

Command:

```
git init
```

Practical project:

```
StudentProject
├── index.html
├── style.css
└── script.js
```

Students initialize the folder as a Git repository.

7. `git status`

Teach students to use:

```
git status
```

Explain:

- Untracked files
- Modified files
- Staged files
- Committed files

Emphasize: `git status` is one of the most useful Git commands.

Students should learn to use it frequently.

8. `git add`

Explain:

- Staging area
- Why files are staged
- Adding one file
- Adding multiple files
- Adding all files

Examples:

```
git add index.html
git add .
```

Demonstrate the difference between:

Working Directory → Staging Area

9. `git commit`

Teach:

- What is a commit?
- Why commits are important
- Commit messages
- Good vs bad commit messages

Example:

```
git commit -m "Added homepage"
```

Explain:

A commit is like saving a meaningful checkpoint of your project.

Module 4: Understanding the Git Workflow

10. Git Workflow

This should be an important practical lesson.

Teach the complete flow:

```
Working Directory
        ↓
    git add
        ↓
Staging Area
        ↓
  git commit
        ↓
Local Repository
        ↓
   git push
        ↓
GitHub Repository
```

Students should understand the difference between:

- Working directory
- Staging area
- Local repository
- Remote repository

Practice

Students modify a file and perform:

```
git status
git add .
git status
git commit -m "Updated project"
git status
```

11. `git log`

Teach:

- Viewing commit history
- Commit ID
- Author
- Date
- Commit message

Command:

```
git log
```

Useful version:

```
git log --oneline
```

Students should learn how to identify previous commits.

Module 5: GitHub Basics

12. Creating a GitHub Account

Teach:

- Creating a GitHub account
- Username
- Profile
- Email verification
- GitHub dashboard
- Repository concept

Activity:
Students create and explore their GitHub profile.

---

13. Creating a GitHub Repository

Teach:

- What is a repository?
- Repository name
- Public vs Private repository
- README
- `.gitignore`
- License

Example repository:

```
student-python-project
```

Module 6: Connecting Git with GitHub

14. Connecting Local Git to GitHub

Explain:

Local Git repository and GitHub repository are two separate repositories. We need to connect them.

Teach:

```
git remote add origin REPOSITORY_URL
```

Check:

```
git remote -v
```

Explain the meaning of:

```
origin
```

---

15. `git remote`

Teach:

```
git remote
```

and:

```
git remote -v
```

Students should understand:

- Remote repository
- Remote name
- `origin`
- Remote URL

---

16. `git push`

Teach:

```
git push
```

and first push:

```
git push -u origin main
```

Explain:

```
Local Repository
        ↓
    git push
        ↓
GitHub Repository
```

Practical:
Students upload their first project to GitHub.

---

17. `git pull`

Teach:

```
git pull
```

Explain:

- Downloading changes from GitHub
- Updating local project
- Difference between `push` and `pull`

```
push = Local → GitHub

pull = GitHub → Local
```

---

18. `git clone`

Teach:

```
git clone REPOSITORY_URL
```

Explain:

- What cloning means
- Downloading an existing repository
- Clone vs download ZIP
- Working with someone else's project

Practical:
Students clone a GitHub repository onto their computer.

Module 7: `.gitignore`

19. Understanding `.gitignore`

Explain:

- Why some files should not be uploaded
- Temporary files
- IDE files
- Python virtual environments
- Passwords/secrets
- Generated files

Example:

```
__pycache__/
*.pyc
.env
venv/
```

Important student lesson:

Never upload passwords, API keys or other sensitive information to GitHub.

Module 8: Git Branches

20. What is a Branch?

Use a simple example:

```
main
 ├── feature-login
 └── feature-payment
```

Explain:

- Why branches are needed
- Main branch
- Feature branch
- Safe development
- Multiple developers working simultaneously

---

21. `git branch`

Teach:

```
git branch
```

Create a branch:

```
git branch feature-login
```

Switching branches should be demonstrated afterward.

22. `git checkout` / `git switch`

Modern approach:

```
git switch feature-login
```

Create and switch:

```
git switch -c feature-login
```

Also explain the older command:

```
git checkout feature-login
```

Students should understand that `git switch` is specifically designed for branch switching, while `git checkout` has broader historical uses.

Module 9: Merging

23. Git Merge

Teach:

- Why branches are merged
- Merging feature branches into main
- Fast-forward merge
- Basic merge workflow

Example:

```
git switch main
```

Then:

```
git merge feature-login
```

Visual explanation:

```
main
 |
 A
 |
 B
 |
 C
      \
       D — E
           ↑
      feature-login
```

After merging, the feature becomes part of the main development line.

Module 10: GitHub Pull Requests

24. Pull Requests

This is one of the most important **GitHub collaboration topics**.

Teach:

- What is a Pull Request?
- Why Pull Requests are used
- Creating a branch
- Making changes
- Pushing branch
- Creating Pull Request
- Reviewing changes
- Approving
- Merging

Basic workflow:

```
Create Branch
      ↓
Make Changes
      ↓
Commit
      ↓
Push
      ↓
Create Pull Request
      ↓
Code Review
      ↓
Merge
```

Module 11: Merge Conflicts

25. Understanding Merge Conflicts

Explain with a simple example.

Two students modify the same line:

```
Student A → "Welcome to GK InfoTech Solutions"

Student B → "Welcome to GK Infotech"
```

Git cannot automatically decide which version should remain.

Teach:

- Why conflicts happen
- How Git displays conflicts
- `<<<<<<<`
- `=======`
- `>>>>>>>`
- Choosing the correct version
- Removing conflict markers
- `git add`
- `git commit`

This should be taught with a **real practical exercise**, not only theory.

Module 12: README Files

26. README.md

Teach:

- What is README?
- Why README is important
- Markdown basics
- Project title
- Project description
- Installation
- Usage
- Features
- Screenshots
- Author information

Example:

```
# Student Management System

A simple Python and MySQL project.

## Features

- Add student
- Update student
- Delete student
- Search student

## Technologies

- Python
- MySQL
```

Students should create a professional README for their own project.

Module 13: GitHub Developer Profile

27. GitHub Profile

Teach students how to use GitHub as a **professional portfolio**.

Topics:

- Professional username
- Profile photo
- Bio
- Skills
- Projects
- Pinned repositories
- README profile
- Contribution activity

Explain:

A GitHub profile can demonstrate what a student can actually build.

Module 14: GitHub Issues

28. GitHub Issues

Teach:

- What is an Issue?
- Reporting bugs
- Feature requests
- Assigning issues
- Labels
- Comments
- Closing issues

Example:

```
Issue #1
Title: Login button not working

Label: Bug
Assigned to: Student A
```

This introduces students to basic software-development practices.

Module 15: GitHub Projects

29. GitHub Projects

Teach:

- Project management
- Tasks
- Issues
- To Do
- In Progress
- Done
- Tracking development

Example:

```
TO DO
 ├── Create database
 └── Design login page

IN PROGRESS
 └── Student registration

DONE
 └── Project setup
```

Students can use this to understand how development teams organize work.

Module 16: Basic GitHub Collaboration

30. GitHub Collaboration

Bring everything together.

Teach students how two or more students can work on one project.

Team workflow

```
GitHub Repository
        ↓
     Clone
        ↓
Create Branch
        ↓
Write Code
        ↓
Commit
        ↓
Push
        ↓
Pull Request
        ↓
Code Review
        ↓
Merge
        ↓
Updated Main Branch
```